Matriosha v2 — ATOMIC_PROMPTS.md

Purpose: Ultra-specific, self-contained prompts for rebuilding the Matriosha CLI via vertical slicing in isolated Abacus AI Agent sessions.

How to use: Copy one task's "Prompt" block verbatim into a new Abacus AI Agent session. Each prompt is designed to be executed in **one session** with **no prior context**. Every prompt references the authoritative docs (`RULES.md` , `SPECIFICATION.md` , `DESIGN.md`) so the agent always re-grounds in the canonical rules.

Repository root (absolute): `/home/ubuntu/github_repos/matriosha`

Vertical slicing principle: Each task delivers a user-observable outcome from CLI entry point → core logic → storage → output. No abstract scaffolding tasks. Each slice is shippable.

Audience:
- 🔧 **Hardcore coders** — exact file paths, function signatures, test expectations, exit codes.
- 🎨 **Vibe coders** — verbatim prompts, copy-paste ready, no interpretation required.

Global agent preamble (embedded in every prompt):

> Before writing any code, read `RULES.md` , `SPECIFICATION.md` , and `DESIGN.md` from the repo root. Obey RULES.md section 1 (CLI-only, Python-only, no web). Enforce SPECIFICATION.md section 4 (binary + base64 + SHA-256 + Merkle) and section 5 (AES-256-GCM + Argon2id). Match DESIGN.md for all user-facing output. Do not invent commands outside the SPECIFICATION.md grammar. Never commit plaintext keys. All file changes must be made on a feature branch and submitted as a PR — never push to `main` .

Phase Index

Phase	Title	Tasks	Est. Sessions
1	Foundation & Setup	P1.1 – P1.3	3
2	Local Mode (Open-Source Core)	P2.1 – P2.7	7
3	Vector Search & Decompression	P3.1 – P3.3	3
4	Managed Mode Layer	P4.1 – P4.6	6
5	Agent Token System	P5.1 – P5.3	3
6	CLI UX & Polish	P6.1 – P6.5	5
7	Testing & Documentation	P7.1 – P7.4	4

Total: 31 atomic sessions.

PHASE 1 — Foundation & Setup

P1.1 — Reset `pyproject.toml` and pin dependencies

Title: Establish canonical `pyproject.toml` with exact dependency pins for Matriosha v2 CLI.

Complexity: Simple

Input files to read (from `/home/ubuntu/github_repos/matriosha`):
- `RULES.md` (sections 1, 2, 3 — security practices)
- `SPECIFICATION.md` (section 6 — repo structure, section 3 — command grammar)
- `pyproject.toml` (current)
- `requirements.txt` (current)

Output files to create/modify:
- `pyproject.toml` (overwrite)
- `.python-version` (create — contents: `3.11`)

Prompt (copy verbatim into new session):

```
You are rebuilding the Matriosha v2 CLI. Repo root: /home/ubuntu/github_repos/matri-
osha.

STEP 1. Read these files in full:
- RULES.md
- SPECIFICATION.md
- DESIGN.md
- pyproject.toml (current)
- requirements.txt (current)

STEP 2. Create a feature branch `phase1/pyproject-reset`.

STEP 3. Overwrite pyproject.toml with a PEP 621 project definition containing EXACTLY:
- [project] name = "matriosha", version = "2.0.0", requires-python = ">=3.11"
- description matching SPECIFICATION.md section 1
- [project.scripts] matriosha = "cli.main:app"
- Dependencies (pinned minor versions):
    typer>=0.12,<0.13
    rich>=13.7,<14
    cryptography>=42,<43
    argon2-cffi>=23.1,<24
    pynacl>=1.5,<2
    requests>=2.32,<3
    httpx>=0.27,<0.28
    supabase>=2.5,<3
    pydantic>=2.7,<3
    numpy>=1.26,<2
    platformdirs>=4.2,<5
    google-cloud-secret-manager>=2.16.0
- [project.optional-dependencies] dev = pytest>=8, pytest-cov>=5, ruff>=0.5, mypy>=1.1
0
- [build-system] using hatchling
- [tool.ruff] line-length=100, target-version="py311"
- [tool.pytest.ini_options] testpaths=["tests"]

STEP 4. Create .python-version with content `3.11`.

STEP 5. DO NOT modify core/ or cli/ in this task.

STEP 6. Run: python -m pip install -e . inside a fresh venv to confirm metadata is val
id (pip install may fail on network — if so, just run `python -c "import tomllib; tom-
llib.loads(open('pyproject.toml','rb').read().decode())"` to validate TOML).

STEP 7. Open a PR titled "phase1: reset pyproject.toml with pinned deps" against
main. Do not merge.

Constraints from RULES.md: no web deps (no next, react, node). Python-only.
```

Success criteria:

- `pyproject.toml` parses as valid TOML.
- `matriosha` script entry points to `cli.main:app`.
- No web/node dependencies present.
- PR opened, not merged.

P1.2 — Canonical directory skeleton with `__init__.py` markers

Title: Lock the repo layout: `core/`, `cli/commands/`, `cli/utils/`, `tests/`, and empty modules for every SPECIFICATION.md command group.

Complexity: Simple

Input files to read:
- `RULES.md` (section 1)
- `SPECIFICATION.md` (section 3 command grammar, section 6 structure)
- existing `cli/` and `core/` trees (inventory only; do not delete)

Output files to create:
- `cli/commands/mode.py`, `auth.py`, `billing.py`, `vault.py`, `memory.py`, `token.py`, `agent.py`, `status.py`, `doctor.py`, `completion.py` (each a stub with `typer.Typer()` app and a TODO docstring pointing to SPECIFICATION.md verbs for that group)
- `cli/utils/output.py` (empty stub with `# DESIGN.md compliance layer`)
- `cli/utils/errors.py` (empty stub with exit-code constants: `EXIT_OK=0`, `EXIT_USAGE=2`, `EXIT_INTEGRITY=10`, `EXIT_AUTH=20`, `EXIT_MODE=30`, `EXIT_NETWORK=40`, `EXIT_UNKNOWN=99`)
- `tests/__init__.py`
- `tests/test_smoke.py` (imports every new module; asserts `True`)

Prompt:

```
Repo root: /home/ubuntu/github_repos/matriosha. Branch: phase1/skeleton.

READ: RULES.md, SPECIFICATION.md, DESIGN.md in full.

TASK: Create the CLI command-group skeleton matching SPECIFICATION.md §3 verbatim.

For each group in §3 (mode, auth, billing, vault, memory, token, agent, status,
doctor, completion):
  Create cli/commands/<group>.py containing:
    import typer
    app = typer.Typer(help="<group description from SPECIFICATION.md>",
no_args_is_help=True)
    # Register one stub subcommand per verb listed in SPECIFICATION.md §3 for this
group.
    # Each stub must:

#   - accept --json/--plain/--verbose/--debug/--profile/--mode global flags (via a
shared dependency from cli/utils/context.py which you will also create)
    #   - raise typer.Exit(code=99) with message "not implemented in phase 1"

Create cli/utils/context.py with a Pydantic model `GlobalContext(mode: Liter-
al['local','managed'], json_output: bool, plain: bool, verbose: bool, debug: bool,
profile: str | None)` and a typer callback that parses global flags.

Create cli/utils/errors.py with exit-code constants EXIT_OK=0, EXIT_USAGE=2,
EXIT_INTEGRITY=10, EXIT_AUTH=20, EXIT_MODE=30, EXIT_NETWORK=40, EXIT_UNKNOWN=99.

Create cli/utils/output.py — empty stub (fill in P6.3).

In cli/main.py wire all command groups via `app.add_typer(<module>.app,
name="<group>")`.

Create tests/test_smoke.py that: from cli.main import app; assert app is not None; and
imports every command-group module.

DO NOT implement any business logic. No crypto. No storage. Just structure.

Run: pytest tests/test_smoke.py — must pass.

Open PR "phase1: CLI skeleton matching SPECIFICATION §3". Do not merge.
```

Success criteria:
- `matriosha --help` lists every group from SPECIFICATION §3.
- `matriosha memory remember` exits with code 99 and stub message.
- `pytest tests/test_smoke.py` passes.

P1.3 — Configuration & profile system

Title: Implement the `--profile` flag, config file at `~/.config/matriosha/config.toml`, and mode persistence (`matriosha mode show/set`).

Complexity: Medium

Input files to read:
- `RULES.md` (section 2 — no plaintext keys; config is non-secret only)
- `SPECIFICATION.md` (§2 dual-mode, §3 mode group)

- `DESIGN.md` (for `mode show` formatting)
- `cli/commands/mode.py` (stub from P1.2)
- `cli/utils/context.py`

Output files:
- `core/config.py` — `load_config()`, `save_config()`, `Profile` pydantic model (fields: `name`, `mode`, `managed_endpoint`, `created_at`); uses `platformdirs.user_config_dir("matriosha")`
- `core/secrets.py` — Google Secret Manager integration (`SecretManager` + `get_secret()` + fallback helpers)
- `cli/commands/mode.py` — fully implemented `show` and `set <local|managed>`
- `tests/test_config.py` — covers profile create, switch, persistence, invalid mode rejection
- `tests/test_secrets.py` — covers missing env vars, missing secret, and local fallback behavior

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase1/config-profiles.

READ: RULES.md, SPECIFICATION.md §2 & §3, DESIGN.md.

GOAL: Persistent config + profiles + `matriosha mode show|set` fully working.


MANDATORY SECRET BASELINE (this becomes the contract for all Phase 4/5 tasks):
0. Add Google Secret Manager wiring now in `core/secrets.py`.
   - Ensure dependency exists in both `pyproject.toml` and `requirements.txt`: `googl
e-cloud-secret-manager>=2.16.0`.
   - Implement this exact pattern (names can differ, behavior cannot):
     ```python
     import os
     from google.api_core.exceptions import GoogleAPICallError, NotFound, Permis-
sionDenied
     from google.cloud import secretmanager

     class SecretManagerError(RuntimeError): ...

     class SecretManager:
         def __init__(self, project_id: str | None = None):
             self.project_id = project_id or os.getenv("GCP_PROJECT_ID")
             self.credentials_path = os.getenv("GOOGLE_APPLICATION_CREDENTIALS")
             self.client = secretmanager.SecretManagerServiceClient() if self.pro-
ject_id else None

         def get_secret(self, secret_name: str, version: str = "latest") -> str |
None:
             if not self.client or not self.project_id:
                 return None
             name = f"projects/{self.project_id}/secrets/{secret_name}/versions/{ver-
sion}"
             try:
                 response = self.client.access_secret_version(request={"name": name})
                 return response.payload.data.decode("utf-8")
             except (NotFound, PermissionDenied, GoogleAPICallError):
                 return None

     def get_secret(secret_name: str, *, default: str | None = None) -> str | None:
         # 1) env override for local dev, 2) GSM, 3) default
         return os.getenv(secret_name) or SecretManager().get_secret(secret_name) or
default
     ```
   - Require and document env vars: `GCP_PROJECT_ID`,
`GOOGLE_APPLICATION_CREDENTIALS`.
   - Error handling requirements:
     - If env vars missing, DO NOT crash local mode.
     - If secret missing/unreadable, return `None` and emit a non-sensitive warning.
   - Fallback requirement: Local mode must continue to work using env/default values
when GSM is unavailable.

1. Create core/config.py:
   - Use platformdirs.user_config_dir("matriosha") → config_dir.
   - Config file: config_dir / "config.toml".
   - Profile model (pydantic): name: str, mode: Literal["local","managed"]="local",
managed_endpoint: str | None=None, created_at: datetime.
   - MatrioshaConfig model: profiles: dict[str, Profile], active_profile: str = "de-
fault".
   - Functions: load_config() -> MatrioshaConfig (creates default profile on first
run), save_config(cfg), get_active_profile(cfg, profile_name_override: str | None) ->
Profile.
```

```
   - Config file must be chmod 0600 after write (RULES.md §2: filesystem hygiene).
   - NEVER store secrets here — only non-sensitive metadata.

2. Implement cli/commands/mode.py:
   - `mode show`: print active profile name + mode. Respect --json (emit {"profile":...
., "mode":...}).
   - `mode set <local|managed>`: validate arg, update active profile, save config.
Print confirmation per DESIGN.md.
   - On invalid mode value: exit code EXIT_USAGE (2).

3. tests/test_config.py with tmp_path + monkeypatch of platformdirs:
   - Fresh dir → default profile created with mode=local.
   - mode set managed → persists across reload.
   - mode set garbage → SystemExit code 2.
   - File permissions 0600 on unix.

4. Ensure `matriosha --profile work mode show` works (profile override via global flag
from cli/utils/context.py).

Constraint: NO network calls. NO auth. NO crypto in this task.

Open PR "phase1: config + profiles + mode show/set".
```

Success criteria:
- `matriosha mode show` prints mode of active profile.
- `matriosha mode set managed && matriosha mode show` → `managed`.
- Config file exists at `~/.config/matriosha/config.toml` with 0600 perms.
- Tests green.

PHASE 2 — Local Mode (Open-Source Core)

P2.1 — Cryptography primitives (AES-256-GCM + Argon2id KDF)

Title: Implement `core/crypto.py` with audited primitives for key derivation, encryption, decryption, and secure random.

Complexity: Complex

Input files:
- `RULES.md` §2 (A02 Cryptographic Failures — MANDATORY parameter minima)
- `SPECIFICATION.md` §5 (Security Requirements)
- existing `core/security.py` (reference only; do not import)

Output files:
- `core/crypto.py`
- `tests/test_crypto.py`

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase2/crypto.

READ FULLY: RULES.md §2.1 (A02), SPECIFICATION.md §5, DESIGN.md (not needed here but
skim).

REFERENCE ONLY (do not inherit code): core/security.py — write crypto.py from scratch.

DELIVER core/crypto.py with this exact public API:

from dataclasses import dataclass

@dataclass(frozen=True)
class KDFParams:
    time_cost: int = 3
    memory_cost: int = 64 * 1024   # 64 MiB in KiB
    parallelism: int = 4
    salt_len: int = 16
    hash_len: int = 32

def generate_salt(length: int = 16) -> bytes: ...        # os.urandom
def derive_key(passphrase: str, salt: bytes, params: KDFParams = KDFParams()) ->
bytes: ...  # Argon2id, returns 32 bytes
def generate_nonce() -> bytes: ...                        # 12 bytes CSPRNG
(os.urandom)
def encrypt(plaintext: bytes, key: bytes, aad: bytes = b"") -> tuple[bytes, bytes]:
    # returns (nonce, ciphertext+tag) using AES-256-GCM (cryptography lib)
def decrypt(nonce: bytes, ct_and_tag: bytes, key: bytes, aad: bytes = b"") -> bytes:
    # raises core.crypto.IntegrityError on auth failure
def new_keypair_ed25519() -> tuple[bytes, bytes]: ...     # (priv, pub) via pynacl
SigningKey for signing vault operations

Custom exceptions: class IntegrityError(Exception), class KDFError(Exception).

HARD RULES (must assert in code):
- KDF time_cost >= 3, memory_cost >= 64*1024, parallelism >= 4 — raise KDFError if
weaker params passed.
- Key length must be exactly 32 bytes.
- Nonce must be exactly 12 bytes.
- No ECB. No non-authenticated modes. AES-GCM only.
- Never log keys, nonces, or plaintext. No __repr__ leaking material.

tests/test_crypto.py must include:
1. Roundtrip encrypt→decrypt returns original plaintext.
2. Tampered ciphertext raises IntegrityError.
3. Wrong AAD raises IntegrityError.
4. derive_key deterministic given same salt+passphrase.
5. Weak KDF params raise KDFError.
6. Nonces from 1000 calls are unique.
7. Ed25519 sign/verify roundtrip.

Use only the `cryptography` package + `argon2-cffi` + `pynacl` + stdlib. No homegrown
crypto.

Open PR "phase2: core crypto primitives (AES-GCM + Argon2id + Ed25519)".
```

Success criteria:

- All 7 tests pass.
- Static check: `rg -n "ECB|DES|MD5|SHA1" core/crypto.py` → no matches.
- KDF params enforced at runtime.

P2.2 — Binary protocol + base64 envelope + SHA-256

Title: Implement `core/binary_protocol.py` — canonical binary memory encoding with SHA-256 block hashing and base64 envelope.

Complexity: Complex

Input files:
- `SPECIFICATION.md` §4 (Memory Data Contract — transport, integrity, mandatory metadata)
- `RULES.md` §2
- `core/crypto.py` (from P2.1)
- existing `core/binary_protocol.py` (reference only)

Output files:
- `core/binary_protocol.py` (rewrite)
- `tests/test_binary_protocol.py`

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase2/binary-protocol.

READ: SPECIFICATION.md §4 in full, RULES.md §2, DESIGN.md.

DEPENDENCY: core/crypto.py already exists (P2.1). Import encrypt/decrypt/gener-
ate_nonce from it.

REWRITE core/binary_protocol.py with this exact public API:

BLOCK_SIZE = 64 * 1024   # 64 KiB fixed

@dataclass
class MemoryEnvelope:
    memory_id: str          # uuid4 str
    mode: Literal["local","managed"]
    encoding: Literal["base64"] = "base64"
    hash_algo: Literal["sha256"] = "sha256"
    merkle_leaves: list[str]     # hex digests per block
    merkle_root: str             # hex digest
    vector_dim: int = 384
    created_at: str              # ISO-8601 UTC
    tags: list[str]
    source: Literal["cli","agent"] = "cli"
    # binary payload handled separately as bytes

def chunk_blocks(plaintext: bytes, block_size: int = BLOCK_SIZE) -> list[bytes]: ...
def block_hash(block: bytes) -> str: ...                # sha256 hex
def merkle_root(leaves: list[str]) -> str: ...          # delegate to core.merkle
def encode_envelope(plaintext: bytes, key: bytes, *, mode: str, tags: list[str], vec-
tor_dim: int = 384, source: str = "cli") -> tuple[MemoryEnvelope, bytes]:
    # 1. chunk, 2. hash each, 3. build merkle, 4. encrypt whole plaintext with AES-
GCM (nonce||ct),
    # 5. base64-encode encrypted payload,
    # 6. return envelope + raw base64 bytes
def decode_envelope(env: MemoryEnvelope, b64_payload: bytes, key: bytes) -> bytes:
    # 1. base64 decode, 2. split nonce/ct, 3. decrypt, 4. re-chunk plaintext, 5. veri-
fy every block hash matches env.merkle_leaves,
    # 6. verify merkle_root, 7. raise IntegrityError on any mismatch
def envelope_to_json(env: MemoryEnvelope) -> str: ...   # matches SPECIFICATION.md §4
.3 exactly
def envelope_from_json(s: str) -> MemoryEnvelope: ...

Metadata JSON MUST contain EXACTLY the keys in SPECIFICATION.md §4.3 (rename merkle_le
aves to merkle_leaf if single-leaf case? NO — §4.3 says merkle_leaf + merkle_root; sto
re list as `merkle_leaves` internally but expose `merkle_leaf` as root-level array
in the JSON for multi-block). Read §4.3 literally and comply.

tests/test_binary_protocol.py:
1. Roundtrip small (<1 block) plaintext.
2. Roundtrip large (5 MB random bytes) — verify all leaves + root.
3. Tampered b64 payload byte → IntegrityError on decode.
4. Tampered one leaf in envelope → IntegrityError.
5. JSON roundtrip produces identical envelope.
6. Empty plaintext → empty leaves + empty-tree root convention (document in code).

Open PR "phase2: binary protocol + SHA-256 + base64 envelope per SPEC §4".
```

Success criteria:

- All tests pass.

- `envelope_to_json` output validated against SPECIFICATION §4.3 keys.
- Tampering detection confirmed.

P2.3 — Merkle tree module

Title: Implement `core/merkle.py` with canonical binary Merkle tree (SHA-256, duplicate-last-on-odd).

Complexity: Medium

Input: `SPECIFICATION.md` §4.2, `core/merkle.py` (reference).

Output: `core/merkle.py` (rewrite), `tests/test_merkle.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase2/merkle.

READ: SPECIFICATION.md §4.2, RULES.md §2.

Public API in core/merkle.py:

def merkle_root(leaves: list[str]) -> str
def merkle_proof(leaves: list[str], index: int) -> list[tuple[str, Literal["L","R"]]]
def verify_proof(leaf: str, proof: list[tuple[str, Literal["L","R"]]], root: str) -> b
ool

Rules:
- All inputs are hex strings of sha256 digests (64 chars).
- Odd number of nodes at any level: duplicate last node (Bitcoin-style).
- Empty leaves → root = sha256(b"").hex() (document).
- Hash concatenation: bytes.fromhex(a)+bytes.fromhex(b) then sha256.

tests/test_merkle.py:
- Known-vector test (hard-code 4 leaves and expected root).
- Odd-count test (3 leaves).
- Proof for every index round-trips via verify_proof.
- Tampered proof fails verification.
- Single-leaf tree root == leaf (or document alternative).

Open PR "phase2: merkle tree module".
```

Success criteria: Tests pass. Binary-protocol tests (P2.2) now use this module successfully.

P2.4 — Local storage backend

Title: Implement `core/storage_local.py` — filesystem-backed encrypted blob + envelope store under `~/.local/share/matriosha/<profile>/`.

Complexity: Medium

Input: `RULES.md` §2, `SPECIFICATION.md` §2.1 & §4, `core/binary_protocol.py`, `core/config.py`.

Output: `core/storage_local.py`, `tests/test_storage_local.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase2/storage-local.

READ: RULES.md §2, SPECIFICATION.md §2.1 + §4, DESIGN.md.

DEPENDENCIES: core.binary_protocol, core.config, platformdirs.

Create core/storage_local.py with this API:

class LocalStore:
    def __init__(self, profile_name: str): ...          # uses platform-
dirs.user_data_dir
    @property
    def root(self) -> Path: ...                         # e.g., ~/.local/share/matri-
osha/<profile>/
    def put(self, env: MemoryEnvelope, b64_payload: bytes) -> Path: ...
        # writes:
        #   root/memories/<memory_id>.env.json   (envelope JSON)
        #   root/memories/<memory_id>.bin.b64    (base64 payload)
        # chmod 0600 both files.
    def get(self, memory_id: str) -> tuple[MemoryEnvelope, bytes]: ...
    def list(self, *, tag: str | None = None, limit: int = 100) ->
list[MemoryEnvelope]: ...
    def delete(self, memory_id: str) -> bool: ...
    def verify(self, memory_id: str, key: bytes) -> bool: ...
        # loads envelope+payload, calls decode_envelope, returns True if integrity
OK; raises IntegrityError otherwise.

Index file: root/index.json maintained atomically (write to index.json.tmp then os.re-
place). Contains {memory_id: {tags, created_at}} for fast list/filter.

HARD RULES:
- All user inputs (memory_id, tags) validated via pydantic/regex ^[A-Za-z0-9_\-:.]{1,1
28}$ — reject others (RULES.md §2 A03 injection via filesystem path traversal).
- Never log file contents.
- Filesystem ops must not follow symlinks (use Path.resolve(strict=False) + check par-
ent).

tests/test_storage_local.py with tmp_path:
1. put+get roundtrip.
2. list filters by tag.
3. delete removes both files + index entry.
4. verify returns True on untouched, raises IntegrityError after on-disk tampering.
5. Path traversal attempt (memory_id="../evil") rejected.
6. Permissions 0600 on unix.

Open PR "phase2: local storage backend".
```

Success criteria: Tests pass. No path traversal possible. Integrity verification works end-to-end.

P2.5 — `matriosha vault init` (key material bootstrap)

Title: Implement `vault init` — generate/derive master key, store encrypted key material locally, wire full CLI-to-storage slice.

Complexity: Complex

Input: `RULES.md` §2, `SPECIFICATION.md` §3 (vault group), `DESIGN.md` (init flow visuals), `core/crypto.py`, `core/config.py`, `core/storage_local.py`.

Output:
- `core/vault.py` (key material management)
- `cli/commands/vault.py` (implement `init` verb only)
- `tests/test_vault_init.py`

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase2/vault-init.

READ: RULES.md §2, SPECIFICATION.md §3 vault group, DESIGN.md (color + banner specs).

GOAL: End-to-end vertical slice — user runs `matriosha vault init` and gets a working
encrypted vault.

1. Create core/vault.py:
   - KEY_FILE = <data_dir>/<profile>/vault.key.enc  (chmod 0600)
   - SALT_FILE = <data_dir>/<profile>/vault.salt    (chmod 0600, 16 bytes)
   - class Vault:
       @classmethod
       def init(cls, profile: str, passphrase: str) -> "Vault":
           # 1. Refuse if KEY_FILE exists (no overwrite unless --force — add --force
to CLI).
           # 2. salt = generate_salt(16).
           # 3. kek = derive_key(passphrase, salt).
           # 4. data_key = os.urandom(32).  # actual memory encryption key
           # 5. nonce,ct = encrypt(data_key, kek).
           # 6. write SALT_FILE, write KEY_FILE as struct: magic b"MTR1" || nonce(12)
|| ct(32+16).
       @classmethod
       def unlock(cls, profile: str, passphrase: str) -> "Vault":

# read salt+encrypted key, derive kek, decrypt data_key. Raise AuthError on wrong
pass.
       data_key: bytes  # only in-memory
   - Never persist data_key in plaintext.

2. Implement cli/commands/vault.py → `init` subcommand:
   - Flags: --force (overwrite), --passphrase (prefer env MATRIOSHA_PASSPHRASE, else t
yper.prompt(hide_input=True, confirmation_prompt=True)).
   - Prints a banner per DESIGN.md on success (exact ASCII from DESIGN.md).
   - --json emits {"status":"ok","profile":..., "salt_file":..., "key_file":...}.
   - Exit codes: 0 ok, 2 usage (missing passphrase in --json mode with no env), 20 aut
h (shouldn't happen in init), 10 integrity (file corruption on --force check).
   - Rate-limit: if 5 failed init attempts within 60s (track via a counter in config
dir), sleep exponentially (RULES.md §2 A07).

3. Other vault verbs (verify/rotate/export/sync) remain stubs — raise "not implemented
in phase 2.5".

4. tests/test_vault_init.py:
   - Happy path: init then unlock with same passphrase succeeds.
   - Wrong passphrase → AuthError.
   - Double init without --force → refuses.
   - --force overwrites.
   - File perms 0600.
   - --json output schema validated.

Open PR "phase2: vault init vertical slice".
```

Success criteria: User can run `matriosha vault init`, get a vault, and subsequent commands can unlock it with the same passphrase.

P2.6 — `matriosha memory remember` (full vertical slice)

Title: Implement `memory remember` — read input, encrypt, hash, merkle, envelope, store locally.

Complexity: Complex

Input: `SPECIFICATION.md` §3 + §4, `DESIGN.md`, `core/vault.py`, `core/binary_protocol.py`, `core/storage_local.py`.

Output: `cli/commands/memory.py` (`remember` only), `tests/test_cmd_remember.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase2/memory-remember.

READ: SPECIFICATION.md §3 memory group + §4 data contract, DESIGN.md, RULES.md.

DEPENDENCIES: core.vault (P2.5), core.binary_protocol (P2.2), core.storage_local (P2.
4).

Implement `matriosha memory remember` in cli/commands/memory.py:

Signature:
  matriosha memory remember [TEXT] [--file PATH] [--tag TAG ...] [--stdin] [--pass-
phrase ...]

Behavior:
1. Resolve input source (exactly one of TEXT / --file / --stdin; else exit 2).
2. Read bytes (UTF-8 for text). Max size 50 MiB → else exit 2 with clear error.
3. Validate tags: each matches ^[a-z0-9][a-z0-9_\-]{0,31}$ → else exit 2.
4. Unlock Vault for active profile using passphrase (env > prompt).
5. Mode check: if active mode == managed, still write locally (managed sync is phase
4).
6. Call encode_envelope(plaintext, vault.data_key, mode=active_mode, tags=tags,
source="cli").
7. LocalStore.put(env, b64).
8. Output:
   - Default: rich panel with memory_id, bytes stored, block count, merkle_root (short
), tags. Per DESIGN.md.
   - --json: {"memory_id":..., "bytes":..., "blocks":..., "merkle_root":..., "tags":
[...], "path":...}.
9. Exit 0 on success.

Error mapping → exit codes (errors.py):
  InvalidInput → 2
  IntegrityError during encode → 10
  AuthError (wrong passphrase) → 20
  Unknown → 99

tests/test_cmd_remember.py using typer.testing.CliRunner + tmp home:
1. remember "hello" → memory_id returned, file exists in store.
2. recall via LocalStore.get + decode_envelope returns "hello".
3. Tag filter invalid → exit 2.
4. File >50MiB → exit 2.
5. Wrong passphrase → exit 20.
6. --json schema validated.
7. 2 remembers produce distinct memory_ids and distinct merkle_roots.

Open PR "phase2: memory remember end-to-end".
```

Success criteria: `matriosha memory remember "hello world" --tag test` → memory persisted, retrievable, integrity-verifiable.

P2.7 — `memory recall`, `memory list`, `memory delete`, `vault verify`

Title: Complete the local-mode read paths: fetch by id, list/filter, delete, and cryptographic vault-wide verify.

Complexity: Medium

Input: all prior Phase 2 outputs.

Output:
- `cli/commands/memory.py` (add `recall`, `list`, `delete`)
- `cli/commands/vault.py` (add `verify`)
- `tests/test_cmd_memory_read.py`, `tests/test_cmd_vault_verify.py`

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase2/memory-read-verify.

READ: SPECIFICATION.md §3 (memory + vault), §4, DESIGN.md, RULES.md.

Implement four commands — each must respect --json/--plain/--verbose.

1. matriosha memory recall <memory_id> [--show-metadata] [--out PATH]
   - Unlock vault, LocalStore.get, decode_envelope (verifies integrity).
   - Default prints plaintext to stdout (binary-safe if --out provided).
   - --show-metadata adds envelope JSON.
   - Not found → exit 2. Integrity failure → exit 10.

2. matriosha memory list [--tag T] [--limit N=50] [--since ISO8601]
   - Calls LocalStore.list. Renders a rich.Table per DESIGN.md with columns: id (short
), created, tags, bytes, merkle_root (short).
   - --json emits array of envelope dicts (minus payload).

3. matriosha memory delete <memory_id> [--yes]
   - Confirmation prompt unless --yes. Returns count deleted.
   - Exit 0 even if not found unless --strict.

4. matriosha vault verify [--deep]
   - Iterates all memories in store. --deep decrypts each and re-verifies merkle.
   - Without --deep: only re-hashes payload and checks leaves+root without decrypt.
   - Output: rich progress bar + summary table. --json: {"total":N,"ok":N,"failed":[{"
id":...,"reason":...}]}.
   - Exit 0 if all ok, 10 if any failure.

Tests:
- test_cmd_memory_read.py: remember → recall matches; list shows it; delete removes
it.
- test_cmd_vault_verify.py: 3 memories all verify; corrupt one payload byte on disk →
verify --deep reports failure with exit 10.

Open PR "phase2: memory read paths + vault verify".
```

Success criteria: Full local CRUD works. Tampering is detected by `vault verify --deep`.

PHASE 3 — Vector Search & Decompression

P3.1 — Local vector embedding + in-memory index

Title: Integrate a deterministic local embedding (default: hash-based or lightweight `sentence-transformers` optional) and build a local vector index for stored memories.

Complexity: Complex

Input: `SPECIFICATION.md` §4 (`vector_dim: 384`), `core/brain.py` (reference), `core/storage_local.py` .

Output: `core/vectors.py` , update to `core/storage_local.py` to store vector sidecar, `tests/test_vectors.py` .

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase3/vectors-local.

READ: SPECIFICATION.md §4 (vector_dim=384), RULES.md §2, DESIGN.md.

Dependencies already declared: numpy. Add sentence-transformers as optional extra `[em
beddings]` in pyproject.toml.

Create core/vectors.py:

class Embedder(Protocol):
    dim: int
    def embed(self, text: str) -> np.ndarray: ...

class HashEmbedder:
    # Deterministic bag-of-hashed-trigrams projected to 384 dims. No external model
required.
    dim = 384
    def embed(self, text): ...   # returns float32 vector, L2-normalized

class SBERTEmbedder:
    # Lazy import; uses "sentence-transformers/all-MiniLM-L6-v2" (384-dim).
    dim = 384
    def embed(self, text): ...

def get_default_embedder() -> Embedder:
    # Env MATRIOSHA_EMBEDDER in {"hash","sbert"}, default "hash" (offline-safe).

class LocalVectorIndex:
    def __init__(self, profile: str): ...    # persists to <data_dir>/<profile>/
vectors.npz + ids.json
    def add(self, memory_id: str, vec: np.ndarray): ...
    def remove(self, memory_id: str): ...
    def search(self, q: np.ndarray, k: int = 10) -> list[tuple[str, float]]: ...  #
cosine sim, descending
    def load(self): ...
    def save(self): ...   # atomic write

Update core/storage_local.py:
- `put` now also accepts `embedding: np.ndarray | None` and stores it in Loc-
alVectorIndex if provided.
- `delete` also removes from index.

Update cli/commands/memory.py `remember` to compute embedding of the plaintext (first
4 KiB if larger) and pass to put().

tests/test_vectors.py:
1. HashEmbedder deterministic: embed("x") == embed("x") and norm ≈ 1.0.
2. LocalVectorIndex add+search returns added id with sim=1.0 on identical query.
3. Persist+reload preserves entries.
4. Remove actually removes.

Open PR "phase3: local vector index + embedders".
```

Success criteria: Remembered memories get indexed; identical text query returns sim ≈ 1.0.

P3.2 — `memory search` and `memory compress`

Title: Semantic search CLI + compress (group similar memories into a parent envelope with references).

Complexity: Medium

Input: P3.1 outputs, `SPECIFICATION.md` §3 memory group.

Output: `cli/commands/memory.py` (`search` , `compress`), `tests/test_cmd_search_compress.py` .

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase3/search-compress.

READ: SPECIFICATION.md §3, DESIGN.md.

1. matriosha memory search <query> [--k 10] [--threshold 0.0] [--tag T]
   - Embed query with default embedder, index.search(k).
   - Filter by tag if provided (post-filter via envelope list).
   - Render table: rank, id, score, created, tags, preview (first 80 chars of decryp-
ted plaintext — decrypt lazily).
   - --json returns list of {memory_id, score, tags, created_at, preview}.

2. matriosha memory compress [--threshold 0.85] [--tag T] [--dry-run]
   - Cluster memories whose pairwise cosine similarity ≥ threshold (simple greedy: pic
k seed, sweep).
   - For each cluster of size ≥ 2:
       - Concatenate plaintexts with separator b"\n---\n".
       - encode_envelope with tags = union + ["compressed"] + ["parent"].
       - Store parent envelope; add child refs in parent metadata extra field `chil-
dren: [memory_ids]` (extend MemoryEnvelope — add optional `children: list[str] |
None`).
       - Do NOT delete children yet — compression is reversible via decompress (P3.3).
   - Output: rich tree showing clusters; --dry-run prints without writing.

tests/test_cmd_search_compress.py:
- Add 5 memories; 3 similar, 2 distinct.
- search returns the 3 together at top.
- compress creates 1 parent with children=[3 ids] and leaves 2 singletons untouched.
- --dry-run creates nothing.

Open PR "phase3: semantic search + compress".
```

Success criteria: Search ranks semantically related items; compress groups them reversibly.

P3.3 — `memory decompress`

Title: Expand a compressed parent back into reconstructed children (when cosine similarity between parent and query/child > 0.9 guard holds).

Complexity: Medium

Input: P3.2 outputs, `SPECIFICATION.md` §3.

Output: `cli/commands/memory.py` (`decompress`), `tests/test_cmd_decompress.py` .

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase3/decompress.

READ: SPECIFICATION.md §3, DESIGN.md, RULES.md.

matriosha memory decompress <parent_id> [--keep-parent] [--min-similarity 0.9]

Behavior:
1. Load parent envelope. If `children` field absent → exit 2 "not a compressed par-
ent".
2. Load all child envelopes (they still exist from compress).
3. For each child: recompute embedding of its plaintext; cosine sim with parent's em-
bedding (stored as sidecar — extend vectors index to support parent vectors).
4. If any child's sim < --min-similarity → refuse entire operation, exit 10
(integrity) with list of failing children.
5. Re-emit each child as an "active" memory (they already exist; mark active in
index).
6. If not --keep-parent: delete parent envelope + payload + index entry.
7. --json: {"restored":[ids],"parent_deleted":bool}.

tests/test_cmd_decompress.py:
- Using fixture from P3.2, decompress parent → 3 children still retrievable, parent re
moved.
- Mutate one child off-manifold (replace plaintext) → sim drops below 0.9 →
decompress refuses with exit 10.
- --keep-parent leaves parent intact.

Open PR "phase3: decompress with 0.9 similarity guard".
```

Success criteria: Round-trip compress→decompress preserves all originals; guard blocks tampered clusters.

PHASE 4 — Managed Mode Layer

P4.1 — Managed client scaffold + Supabase project bootstrap doc

Title: Create `core/managed/client.py` — thin async HTTP client to a configured managed endpoint, plus `docs/MANAGED_BOOTSTRAP.md` describing required Supabase schema (tables, RLS, pgvector).

Complexity: Medium

Input: `RULES.md` §2, `SPECIFICATION.md` §2.2 + §5, `DESIGN.md`.

Output: `core/managed/__init__.py`, `core/managed/client.py`, `core/managed/schema.sql`, `docs/MANAGED_BOOTSTRAP.md`, `tests/test_managed_client.py` (with httpx mock).

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase4/managed-client.

READ: RULES.md §2 (A01 access control, A03 injection), SPECIFICATION.md §2.2 & §5, DE
SIGN.md.


GSM REQUIREMENT (mandatory for this task):
- Add/verify dependency: `google-cloud-secret-manager>=2.16.0` in touched dependency m
anifests.
- Reuse `core/secrets.py::SecretManager` from P1.3; if missing, implement it first us-
ing `google.cloud.secretmanager.SecretManagerServiceClient`.
- Read secrets by name (never hardcode): `SUPABASE_URL`,
`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHO
OK_SECRET`, and task-specific keys.
- Secret lookup order must be deterministic: `os.getenv(NAME)` → Google Secret Man-
ager (`projects/{GCP_PROJECT_ID}/secrets/{NAME}/versions/latest`) → safe local fall-
back (`None` or explicit local default).
- Required environment variables for GSM access: `GCP_PROJECT_ID`, `GOOGLE_APPLICATIO
N_CREDENTIALS`.
- Missing secret handling: raise actionable errors for managed mode paths, but keep lo
cal mode functional (fail open to local mode, fail closed for managed actions that re-
quire the secret).
- Never log secret values, secret payloads, or full credential paths; log only secret
names and high-level cause.

1. Create core/managed/schema.sql defining Supabase tables:
   - users (id uuid pk references auth.users)
   - profiles (user_id uuid, name text, created_at timestamptz)
   - memories (id uuid pk, user_id uuid fk, envelope jsonb, payload_b64 text, cre-
ated_at timestamptz, tags text[])
   - memory_vectors (memory_id uuid pk fk memories, embedding vector(384))
   - subscriptions (user_id uuid pk, status text, current_period_end timestamptz, stri
pe_customer_id text, stripe_subscription_id text, plan_code text, unit_price_cents
int, agent_quota int, storage_cap_bytes bigint)
   - agent_tokens (id uuid pk, user_id uuid fk, token_hash text unique, name text, cre
ated_at, revoked_at)
   Enable RLS on every table with policies:
     memories/memory_vectors/profiles: user_id = auth.uid()
     agent_tokens: user_id = auth.uid()
     subscriptions: user_id = auth.uid()
   CREATE EXTENSION vector; CREATE INDEX on memory_vectors USING ivfflat (embedding ve
ctor_cosine_ops).

2. Create core/managed/client.py:
   - Initialize runtime secrets once at startup using `from core.secrets import get_se
cret`:
     - `supabase_url = get_secret("SUPABASE_URL")`
     - `supabase_service_role_key = get_secret("SUPABASE_SERVICE_ROLE_KEY")`
     - `stripe_secret_key = get_secret("STRIPE_SECRET_KEY")`
   - If any of the above are missing and mode is managed, raise a typed config error w
ith remediation text: "Set env or create GSM secret <NAME>".
   class ManagedClient:
       def __init__(self, endpoint: str, access_token: str): ...
       # All methods async via httpx.AsyncClient, 10s timeout, retries with
exponential backoff (max 3) on 5xx only.
       async def whoami(self) -> dict
       async def upload_memory(self, envelope: dict, payload_b64: str, embedding: list
[float]) -> str  # returns server id
       async def fetch_memory(self, memory_id: str) -> tuple[dict, str]   # envelope,
b64
       async def list_memories(self, *, tag=None, limit=50) -> list[dict]
       async def delete_memory(self, memory_id: str) -> bool
```

```
        async def search(self, embedding: list[float], k=10) -> list[dict]
        async def get_subscription(self) -> dict
        async def start_checkout(self, plan: str = "eur_monthly", quantity: int = 1) -
> dict   # returns checkout_url; quantity counts 3-agent pricing blocks
        async def cancel_subscription(self) -> dict
        async def create_agent_token(self, name: str) -> dict   # returns {id,
token_plaintext (one-time)}
        async def revoke_agent_token(self, token_id: str) -> bool
        async def list_agent_tokens(self) -> list[dict]
    - All requests send Authorization: Bearer <token>.
    - Body serialization uses json.dumps with sort_keys=True, ensure_ascii=False.
    - NEVER log headers or bodies (RULES.md §2.1 A09).

3. Create docs/MANAGED_BOOTSTRAP.md walking an operator through:
    a. Create Supabase project.
    b. Run schema.sql in SQL editor.
    c. Enable Vault extension.
    d. Configure Stripe webhook → Edge Function that updates subscriptions table (statu
s, quantity-derived agent_quota, storage_cap_bytes).
    e. Set env vars for managed backend.
    (Bullet list + copy-paste SQL. No web UI instructions since RULES.md §1 excludes we
b work — this is ops doc only.)

4. tests/test_managed_client.py with respx mocking:
    - whoami happy path.
    - 500 → retries 3× then fails with NetworkError.
    - upload_memory sends expected JSON keys.
    - Auth failure (401) → raises AuthError (no retry).

Open PR "phase4: managed client + schema + bootstrap doc".
```

Success criteria: Client class covers all managed ops needed by P4.2–P5; schema deployable to Supabase.

P4.2 — `matriosha auth login` / `logout` / `whoami` (device-code CLI-native flow)

Title: Implement CLI-native authentication (OAuth 2.0 device authorization grant — no browser OAuth in-repo per RULES.md §1).

Complexity: Complex

Input: `RULES.md` §1 (no browser OAuth) and §2.1 A07, `SPECIFICATION.md` §3 auth group, `DESIGN.md`.

Output: `core/managed/auth.py`, `cli/commands/auth.py`, `tests/test_cmd_auth.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase4/auth-device-code.

READ: RULES.md §1 & §2.1 A07, SPECIFICATION.md §3 auth group, DESIGN.md.


GSM REQUIREMENT (mandatory for this task):
- Add/verify dependency: `google-cloud-secret-manager>=2.16.0` in touched dependency m
anifests.
- Reuse `core/secrets.py::SecretManager` from P1.3; if missing, implement it first us-
ing `google.cloud.secretmanager.SecretManagerServiceClient`.
- Read secrets by name (never hardcode): `SUPABASE_URL`,
`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHO
OK_SECRET`, and task-specific keys.
- Secret lookup order must be deterministic: `os.getenv(NAME)` → Google Secret Man-
ager (`projects/{GCP_PROJECT_ID}/secrets/{NAME}/versions/latest`) → safe local fall-
back (`None` or explicit local default).
- Required environment variables for GSM access: `GCP_PROJECT_ID`, `GOOGLE_APPLICATIO
N_CREDENTIALS`.
- Missing secret handling: raise actionable errors for managed mode paths, but keep lo
cal mode functional (fail open to local mode, fail closed for managed actions that re-
quire the secret).
- Never log secret values, secret payloads, or full credential paths; log only secret
names and high-level cause.

IMPORTANT: Per RULES.md §1, this repo forbids browser OAuth UI. We use the OAuth 2.0 *
*device authorization grant** (RFC 8628) — device displays a code, user enters it on t
heir own device's browser (which is NOT this repo). The CLI only polls.

1. core/managed/auth.py:
   class DeviceCodeFlow:
       async def start(self, endpoint) -> dict:   # POST /oauth/device → user_code,
device_code, verification_uri, interval, expires_in
       async def poll(self, device_code, endpoint, interval) -> dict:   # returns {ac-
cess_token, refresh_token, expires_at} on success
   class TokenStore:
       # stores tokens at <data_dir>/<profile>/tokens.enc (AES-GCM with key derived
from user passphrase — reuse vault KEK? NO — use a separate per-profile auth key file
encrypted at rest with OS keyring if available, else passphrase prompt).
       def save(tokens): ...
       def load() -> dict | None: ...
       def clear(): ...

2. cli/commands/auth.py:
   - login: start device flow → print rich panel with user_code (BIG) + verifica-
tion_uri per DESIGN.md → poll with spinner → store tokens. --json emits
{"status":"ok","user_code":...,"verification_uri":...} during start phase plus final
{"status":"authenticated"} on success.
   - logout: clear TokenStore + revoke on server (best-effort).
   - whoami: ManagedClient.whoami() + render email/user_id/subscription status.
   - switch: list profiles, pick active (already in P1.3 — just ensure it re-reads tok
ens).

   Rate-limiting: login attempts > 5 in 60s per profile → exponential backoff (RULES.m
d §2.1 A07).

3. Managed-only guard: auth commands in `local` mode → exit 30 with "auth requires --
mode managed or `mode set managed`".

4. tests/test_cmd_auth.py with respx + monkeypatched token store:
   - Happy device flow.
   - Timeout (expires_in exceeded) → exit 20.
   - Rate-limit backoff trips after 5 failures.
```

```
    - logout clears store.
    - whoami in local mode → exit 30.

Open PR "phase4: device-code auth login/logout/whoami".
```

Success criteria: `matriosha auth login` completes fully via device code; `whoami` returns server identity.

P4.3 — Managed storage sync (`vault sync` , `vault export`)

Title: Implement bidirectional sync of encrypted envelopes to Supabase + export to portable archive.

Complexity: Complex

Input: P4.1, P4.2, `core/storage_local.py` , `SPECIFICATION.md` §3 vault group + §4, `RULES.md` §2 (A01).

Output: `core/managed/sync.py` , `cli/commands/vault.py` (`sync` , `export`), `tests/test_cmd_sync.py` .

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase4/vault-sync.

READ: SPECIFICATION.md §3 vault + §4, RULES.md §2 (A01 + A02 — data_key never up-
loaded in plaintext).


GSM REQUIREMENT (mandatory for this task):
- Add/verify dependency: `google-cloud-secret-manager>=2.16.0` in touched dependency m
anifests.
- Reuse `core/secrets.py::SecretManager` from P1.3; if missing, implement it first us-
ing `google.cloud.secretmanager.SecretManagerServiceClient`.
- Read secrets by name (never hardcode): `SUPABASE_URL`,
`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHO
OK_SECRET`, and task-specific keys.
- Secret lookup order must be deterministic: `os.getenv(NAME)` → Google Secret Man-
ager (`projects/{GCP_PROJECT_ID}/secrets/{NAME}/versions/latest`) → safe local fall-
back (`None` or explicit local default).
- Required environment variables for GSM access: `GCP_PROJECT_ID`, `GOOGLE_APPLICATIO
N_CREDENTIALS`.
- Missing secret handling: raise actionable errors for managed mode paths, but keep lo
cal mode functional (fail open to local mode, fail closed for managed actions that re-
quire the secret).
- Never log secret values, secret payloads, or full credential paths; log only secret
names and high-level cause.

HARD RULE: Only ENCRYPTED envelopes + encrypted payloads travel to the server. The dat
a_key stays local UNLESS the user opts into Supabase Vault storage (see P4.4).

1. core/managed/sync.py:
   class SyncEngine:
       def __init__(self, local: LocalStore, remote: ManagedClient, embedder: Embed-
der): ...
       async def push(self, *, since: datetime | None = None) -> SyncReport
       async def pull(self, *, since: datetime | None = None) -> SyncReport
       async def sync(self) -> SyncReport     # push then pull, deterministic
   - Server id ↔ local id mapping stored in <data_dir>/<profile>/sync_state.json.
   - Conflict resolution: last-writer-wins on created_at, but envelope immutability me
ans no in-place edits — conflicts shouldn't occur; log anomalies.
   - Every upload: envelope JSON + base64 payload + embedding (float32 list). Verify s
erver round-trip hash matches before marking synced.

2. cli/commands/vault.py additions:
   - sync: runs SyncEngine.sync. Rich progress bars. --json report.
   - export [--out PATH.tar.gz]: pack local memories + envelope index + manifest into
a tar.gz. Does NOT include data_key. Manifest contains merkle_root of the whole archiv
e (merkle over memory merkle roots).

3. Managed-only: sync requires mode=managed + valid token. Export works in both modes.

4. tests/test_cmd_sync.py with respx-mocked ManagedClient:
   - 5 local memories → push → server sees 5.
   - Idempotent: second push is no-op.
   - Server has 3 additional memories → pull adds them locally with merkle verified.
   - Payload tampered server-side → pull rejects with exit 10.
   - export produces tar.gz with expected manifest.

Open PR "phase4: managed sync + portable export".
```

Success criteria: `matriosha vault sync` produces consistent server+local state; export archive round-trips.

P4.4 — Supabase Vault integration for key storage (opt-in)

Title: Add `vault rotate` and optional “managed key custody” using Supabase Vault (pgsodium) to store the encrypted data_key.

Complexity: Complex

Input: `RULES.md` §2 (A02), `SPECIFICATION.md` §2.2 + §5, P4.1 schema.

Output: update `core/managed/schema.sql` (vault_keys table + pgsodium secrets), `core/managed/key_custody.py`, `cli/commands/vault.py` (`rotate`), `tests/test_key_custody.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase4/key-custody.

READ: RULES.md §2.1 A02, SPECIFICATION.md §2.2 + §5.


GSM REQUIREMENT (mandatory for this task):
- Add/verify dependency: `google-cloud-secret-manager>=2.16.0` in touched dependency m
anifests.
- Reuse `core/secrets.py::SecretManager` from P1.3; if missing, implement it first us-
ing `google.cloud.secretmanager.SecretManagerServiceClient`.
- Read secrets by name (never hardcode): `SUPABASE_URL`,
`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHO
OK_SECRET`, and task-specific keys.
- Secret lookup order must be deterministic: `os.getenv(NAME)` → Google Secret Man-
ager (`projects/{GCP_PROJECT_ID}/secrets/{NAME}/versions/latest`) → safe local fall-
back (`None` or explicit local default).
- Required environment variables for GSM access: `GCP_PROJECT_ID`, `GOOGLE_APPLICATIO
N_CREDENTIALS`.
- Missing secret handling: raise actionable errors for managed mode paths, but keep lo
cal mode functional (fail open to local mode, fail closed for managed actions that re-
quire the secret).
- Never log secret values, secret payloads, or full credential paths; log only secret
names and high-level cause.

IMPORTANT (RULES.md): plaintext data_key NEVER leaves the client. We store a second-wr
apped copy in Supabase Vault, wrapped with a server-side pgsodium key AND the
client's passphrase-derived KEK.

1. Extend core/managed/schema.sql:
   - Table vault_keys (user_id uuid pk, wrapped_key bytea, kdf_salt bytea, algo text d
efault 'aes-gcm', rotated_at timestamptz).
   - RLS: user_id = auth.uid().
   - Edge function (documented in docs/MANAGED_BOOTSTRAP.md) wraps/unwraps via pgsodi-
um on request.

2. core/managed/key_custody.py:
   async def upload_wrapped_key(remote, kek_salt, wrapped_key_bytes) -> None
   async def fetch_wrapped_key(remote) -> tuple[salt, wrapped_key]
   def double_wrap(data_key, kek, server_pubkey) -> bytes   # AES-GCM wrap + sealed
box via pynacl
   def double_unwrap(blob, kek, server_privkey_ref) -> bytes
   (If project chooses pgsodium-only, implement server-side unwrap via RPC.)

3. cli/commands/vault.py `rotate`:
   - In local mode: re-derive KEK from new passphrase, re-encrypt data_key locally,
re-encrypt every memory? NO — memories are encrypted with data_key which stays the sam
e. Only the KEK wrapping changes. Confirm in code comment.
   - In managed mode: also upload new wrapped_key.
   - --rotate-data-key flag: generate new data_key, re-encrypt every local memory
(and push in managed mode). Requires --confirm-bulk. Uses temporary directory then ato
mic swap.

4. tests/test_key_custody.py:
   - double_wrap/unwrap roundtrip.
   - Rotate KEK only: old memories still decrypt after.
   - Rotate --rotate-data-key: old on-disk ciphertext replaced atomically; mid-flight
crash simulation (raise after N memories) leaves both sets valid (use a marker file; o
n next run, resume).
   - Managed mode: server receives new wrapped_key.

Open PR "phase4: supabase vault key custody + vault rotate".
```

Success criteria: Keys can be rotated safely; optional server custody works with RLS.

P4.5 — `matriosha billing` (subscribe / status / cancel) — scalable €9 per 3 agents

Title: Stripe-backed subscription CLI gated behind managed mode.

Complexity: Medium

Input: `RULES.md` §1 & §2.1 A01, `SPECIFICATION.md` §3 billing group, `DESIGN.md`, P4.1 client.

Output: `cli/commands/billing.py`, `tests/test_cmd_billing.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase4/billing.

READ: SPECIFICATION.md §3 billing, RULES.md §1 (no browser OAuth, but checkout_url is
acceptable — CLI prints URL, user opens on own device), DESIGN.md.


GSM REQUIREMENT (mandatory for this task):
- Add/verify dependency: `google-cloud-secret-manager>=2.16.0` in touched dependency m
anifests.
- Reuse `core/secrets.py::SecretManager` from P1.3; if missing, implement it first us-
ing `google.cloud.secretmanager.SecretManagerServiceClient`.
- Read secrets by name (never hardcode): `SUPABASE_URL`,
`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHO
OK_SECRET`, and task-specific keys.
- Secret lookup order must be deterministic: `os.getenv(NAME)` → Google Secret Man-
ager (`projects/{GCP_PROJECT_ID}/secrets/{NAME}/versions/latest`) → safe local fall-
back (`None` or explicit local default).
- Required environment variables for GSM access: `GCP_PROJECT_ID`, `GOOGLE_APPLICATIO
N_CREDENTIALS`.
- Missing secret handling: raise actionable errors for managed mode paths, but keep lo
cal mode functional (fail open to local mode, fail closed for managed actions that re-
quire the secret).
- Never log secret values, secret payloads, or full credential paths; log only secret
names and high-level cause.

Canonical pricing model: base €9/month includes 3 agents + 3 GB managed storage.
Every additional 3 agents adds +€9/month and +3 GB storage. Currency EUR.
Stripe catalog expectation:
- Base plan id: "matriosha_base_3_agents_eur_900_monthly"
- Add-on plan id: "matriosha_addon_3_agents_eur_900_monthly"

1. cli/commands/billing.py:
   - Resolve Stripe credentials via `get_secret("STRIPE_SECRET_KEY")` and `get_secret
("STRIPE_WEBHOOK_SECRET")` (never literal values).
   - If Stripe secrets are unavailable: managed billing commands exit 40 with action-
able remediation, but local mode remains unaffected.
   - status: ManagedClient.get_subscription() → rich panel with plan, status (active/t
rialing/past_due/canceled), current_period_end, renews_on, `agent_quota`, `agent_in_u
se`, `storage_cap_bytes`, `storage_used_bytes`. --json emits raw.
   - subscribe: support `--agent-pack-count <n>` (default 1 where 1 = base block of 3
agents).
     - Compute expected quota/cap as:
       - `agent_quota = 3 * n`
       - `storage_cap_bytes = (3 * n) * 1024^3`
       - `monthly_price_eur = 9 * n`
     - Call ManagedClient.start_checkout("eur_monthly", quantity=n) so Stripe
quantity maps to 3-agent blocks.
     - Server returns Stripe Checkout URL → CLI prints URL + QR code (use `qrcode` li
b — add as extra dep) → polls subscription status until active or 10 minutes elapsed
→ shows confirmation with total monthly price, agent quota, and storage cap.
     - --json emits {"checkout_url":..., "status":"pending","agent_pack_count":n} and
on completion {"status":"active","agent_quota":...,"storage_cap_bytes":...,"monthly_pr
ice_eur":...}.
   - cancel: requires --yes confirmation; calls ManagedClient.cancel_subscription. Not
e: cancellation is at period end; display next renewal as cancellation date.

2. Add `qrcode[pil]` to [project.optional-dependencies] cli-ux. Import lazily.

3. Managed-only guard. local mode → exit 30.

4. tests/test_cmd_billing.py with respx:
   - status active/past_due/canceled rendered differently and includes quota/cap field
```

```
s.
   - subscribe with default pack count (n=1) shows URL + reaches "active" via simu-
lated polling.
   - subscribe with `--agent-pack-count 3` computes €27/month, 9-agent quota, and 9 G
B cap.
   - invalid pack count (`0`, negative, non-int) exits with code 2.
   - cancel refuses without --yes.

Open PR "phase4: billing subscribe/status/cancel with scalable €9-per-3-agent pri-
cing".
```

Success criteria: User completes a Stripe checkout via URL and CLI reflects active subscription with correct monthly price, agent quota, and storage cap.

P4.6 — Managed-mode enforcement + automated background sync

Title: Gate every managed-only command, implement `matriosha vault sync --watch` (polling loop) and `--auto` hook into remember/delete.

Complexity: Medium

Input: P4.2 / P4.3 / P4.5, `RULES.md` §1 (managed must not break local).

Output: `cli/utils/mode_guard.py`, update `cli/commands/vault.py` + `memory.py`, `tests/test_mode_guard.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase4/mode-guard-autosync.

READ: RULES.md §1 & §3, SPECIFICATION.md §2 & §3.


GSM REQUIREMENT (mandatory for this task):
- Add/verify dependency: `google-cloud-secret-manager>=2.16.0` in touched dependency m
anifests.
- Reuse `core/secrets.py::SecretManager` from P1.3; if missing, implement it first us-
ing `google.cloud.secretmanager.SecretManagerServiceClient`.
- Read secrets by name (never hardcode): `SUPABASE_URL`,
`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHO
OK_SECRET`, and task-specific keys.
- Secret lookup order must be deterministic: `os.getenv(NAME)` → Google Secret Man-
ager (`projects/{GCP_PROJECT_ID}/secrets/{NAME}/versions/latest`) → safe local fall-
back (`None` or explicit local default).
- Required environment variables for GSM access: `GCP_PROJECT_ID`, `GOOGLE_APPLICATIO
N_CREDENTIALS`.
- Missing secret handling: raise actionable errors for managed mode paths, but keep lo
cal mode functional (fail open to local mode, fail closed for managed actions that re-
quire the secret).
- Never log secret values, secret payloads, or full credential paths; log only secret
names and high-level cause.

1. cli/utils/mode_guard.py:
   def require_mode(required: Literal["local","managed"]):
       # typer dependency. If active mode mismatches, exit code 30 with message:
       # "this command requires <required> mode; run `matriosha mode set <required>`"

2. Apply guards:
   - managed-only: auth.*, billing.*, vault sync, token.*, agent.*.
   - local-always-ok: memory.*, vault init/verify/export/rotate, mode.*, status, doc-
tor.

3. Implement vault sync --watch INTERVAL_SECONDS (default 60): loops SyncEngine.sync,
respects SIGINT. Logs per iteration. --json streams NDJSON per sync report.

4. In cli/commands/memory.py remember/delete: if mode==managed and config.managed.auto
_sync==True, schedule a fire-and-forget sync after local write (use asyncio.cre-
ate_task within a short-lived event loop; ensure errors logged but not crashing). Con-
figurable via `matriosha config set managed.auto_sync true/false` — extend mode.py wi
th a generic set/get subcommand OR add minimal config command (keep scope tight: just
auto_sync boolean for now).

5. tests/test_mode_guard.py:
   - billing status in local mode → exit 30.
   - memory remember works in both modes.
   - --watch cancels cleanly on SIGINT.
   - auto_sync true: remember triggers one sync call.

Open PR "phase4: mode guards + autosync".
```

Success criteria: Managed-only commands fail-fast in local mode; autosync demonstrably pushes new memories.

PHASE 5 — Agent Token System

P5.1 — `token generate` / `token list` / `token revoke` / `token inspect`

Title: CLI token lifecycle against managed backend (RLS-enforced, hashed at rest).

Complexity: Medium

Input: `RULES.md` §2.1 A01+A07, `SPECIFICATION.md` §3 token group, P4.1 client/schema.

Output: `cli/commands/token.py`, `tests/test_cmd_token.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase5/tokens.

READ: SPECIFICATION.md §3 token group, RULES.md §2.1 A01 + A07, DESIGN.md.


GSM REQUIREMENT (mandatory for this task):
- Add/verify dependency: `google-cloud-secret-manager>=2.16.0` in touched dependency m
anifests.
- Reuse `core/secrets.py::SecretManager` from P1.3; if missing, implement it first us-
ing `google.cloud.secretmanager.SecretManagerServiceClient`.
- Read secrets by name (never hardcode): `SUPABASE_URL`,
`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHO
OK_SECRET`, and task-specific keys.
- Secret lookup order must be deterministic: `os.getenv(NAME)` → Google Secret Man-
ager (`projects/{GCP_PROJECT_ID}/secrets/{NAME}/versions/latest`) → safe local fall-
back (`None` or explicit local default).
- Required environment variables for GSM access: `GCP_PROJECT_ID`, `GOOGLE_APPLICATIO
N_CREDENTIALS`.
- Missing secret handling: raise actionable errors for managed mode paths, but keep lo
cal mode functional (fail open to local mode, fail closed for managed actions that re-
quire the secret).
- Never log secret values, secret payloads, or full credential paths; log only secret
names and high-level cause.

HARD RULES:
- Server stores ONLY sha256(token) + salt, never plaintext.
- Token shown ONCE at creation — never recoverable.
- Token format: "mt_" + 32 bytes url-safe base64 (= ~43 chars after base64).

1. cli/commands/token.py:
   - Before token operations, validate backend credentials via `get_secret("SUPA-
BASE_URL")` and `get_secret("SUPABASE_SERVICE_ROLE_KEY")`.
   - generate <name> [--scope read|write|admin (default write)] [--expires DURATION]
     → ManagedClient.create_agent_token(name, scope, expires_at) → server returns
{id, token_plaintext}
     → Rich panel: "STORE THIS TOKEN NOW — it will not be shown again." + the token
on its own line.
     → --json emits {"id":...,"token":..., "name":...,"scope":...,"expires_at":...}.
   - list → table of id, name, scope, created_at, last_used, expires_at, revoked.
   - revoke <id_or_prefix> [--yes]
   - inspect <id_or_prefix> → full token metadata (no plaintext).

2. Rate-limit generation to 10/hour per user (server enforces; client surfaces 429 mes
sage with exit 40).

3. Managed-only guard via require_mode("managed").

4. tests/test_cmd_token.py with respx:
   - generate returns token; listed with revoked=false.
   - revoke then list shows revoked=true.
   - inspect with prefix works.
   - 429 → exit 40.

Open PR "phase5: agent token lifecycle CLI".
```

Success criteria: Token issued once, stored securely, listable, revocable.

P5.2 — Agent authentication surface (`agent connect` / `agent list` / `agent remove`)

Title: Register an external agent against a token; persist agent metadata; expose via CLI.

Complexity: Medium

Input: P5.1, `SPECIFICATION.md` §3 agent group.

Output: `core/managed/agents.py`, `cli/commands/agent.py`, `tests/test_cmd_agent.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase5/agents.

READ: SPECIFICATION.md §3 agent group, RULES.md §2.1.


GSM REQUIREMENT (mandatory for this task):
- Add/verify dependency: `google-cloud-secret-manager>=2.16.0` in touched dependency m
anifests.
- Reuse `core/secrets.py::SecretManager` from P1.3; if missing, implement it first us-
ing `google.cloud.secretmanager.SecretManagerServiceClient`.
- Read secrets by name (never hardcode): `SUPABASE_URL`,
`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHO
OK_SECRET`, and task-specific keys.
- Secret lookup order must be deterministic: `os.getenv(NAME)` → Google Secret Man-
ager (`projects/{GCP_PROJECT_ID}/secrets/{NAME}/versions/latest`) → safe local fall-
back (`None` or explicit local default).
- Required environment variables for GSM access: `GCP_PROJECT_ID`, `GOOGLE_APPLICATIO
N_CREDENTIALS`.
- Missing secret handling: raise actionable errors for managed mode paths, but keep lo
cal mode functional (fail open to local mode, fail closed for managed actions that re-
quire the secret).
- Never log secret values, secret payloads, or full credential paths; log only secret
names and high-level cause.

Server-side schema extension (append to core/managed/schema.sql):
  agents table: id uuid pk, user_id uuid fk, token_id uuid fk agent_tokens, name
text, agent_kind text, fingerprint text, connected_at, last_seen.
  RLS user_id = auth.uid().

1. core/managed/agents.py wraps:
   async def connect(remote, token_plaintext, name, agent_kind) -> dict
       # uses provided token as Bearer for this single request to POST /agents/connect
   async def list_agents(remote) -> list
   async def remove_agent(remote, agent_id) -> bool

2. cli/commands/agent.py:
   - connect --token TOKEN --name NAME --kind KIND
     → calls connect, prints fingerprint + agent_id.
     → SECURITY: token arg is prompted (hidden) if not provided; never logged.
   - list
   - remove <id_or_prefix> [--yes]

3. Managed-only guard.

4. tests/test_cmd_agent.py:
   - connect with valid token → agent registered.
   - invalid token → exit 20.
   - remove idempotent.

Open PR "phase5: agent connect/list/remove".
```

Success criteria: Agents appear in server list after connect; revoking token also invalidates agent.

P5.3 — Token-scope enforcement in managed handlers

Title: Client-side scope hinting + server-side RLS policies wired up.

Complexity: Medium

Input: P5.1, P5.2, `RULES.md` §2.1 A01.

Output: update `core/managed/schema.sql` (scope check functions), `core/managed/client.py` (scope-aware error mapping), `tests/test_scope_enforcement.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase5/scope-enforcement.

READ: RULES.md §2.1 A01, SPECIFICATION.md §3 token + §5.


GSM REQUIREMENT (mandatory for this task):
- Add/verify dependency: `google-cloud-secret-manager>=2.16.0` in touched dependency m
anifests.
- Reuse `core/secrets.py::SecretManager` from P1.3; if missing, implement it first us-
ing `google.cloud.secretmanager.SecretManagerServiceClient`.
- Read secrets by name (never hardcode): `SUPABASE_URL`,
`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHO
OK_SECRET`, and task-specific keys.
- Secret lookup order must be deterministic: `os.getenv(NAME)` → Google Secret Man-
ager (`projects/{GCP_PROJECT_ID}/secrets/{NAME}/versions/latest`) → safe local fall-
back (`None` or explicit local default).
- Required environment variables for GSM access: `GCP_PROJECT_ID`, `GOOGLE_APPLICATIO
N_CREDENTIALS`.
- Missing secret handling: raise actionable errors for managed mode paths, but keep lo
cal mode functional (fail open to local mode, fail closed for managed actions that re-
quire the secret).
- Never log secret values, secret payloads, or full credential paths; log only secret
names and high-level cause.

1. schema.sql additions:
   - Function check_token_scope(required text) returns boolean (reads current request
JWT claim "scope").
   - RLS policies on memories:
       SELECT if check_token_scope('read') OR check_token_scope('write') OR check_toke
n_scope('admin')
       INSERT/UPDATE if check_token_scope('write') OR check_token_scope('admin')
       DELETE if check_token_scope('admin')

2. core/managed/client.py: on 403 responses, parse server error code "insuffi-
cient_scope" → raise ScopeError(scope_required, scope_provided). Map to exit 20.

3. tests/test_scope_enforcement.py with respx:
   - read-scope token + delete_memory → exit 20 with helpful message.
   - admin token → all ops succeed.

Open PR "phase5: token scope enforcement end-to-end".
```

Success criteria: Scopes observably restrict operations; errors are actionable.

PHASE 6 — CLI UX & Polish

P6.1 — Interactive launcher (Daytona-style TUI menu)

Title: `matriosha` with no args launches a rich interactive menu (arrow-key navigation, search, help footer).

Complexity: Complex

Input: `DESIGN.md` (full file), `SPECIFICATION.md` §3, existing stubs.

Output: `cli/tui/launcher.py`, update `cli/main.py`, `tests/test_tui_launcher.py` (smoke).

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase6/launcher.

READ: DESIGN.md IN FULL, SPECIFICATION.md §3.

Dependency: add `textual>=0.70,<1` to [project.optional-dependencies] tui; fallback
to rich.prompt.Prompt + questionary (add questionary>=2.0) if textual too heavy. Pick
questionary for simplicity (CI-friendly).

1. cli/tui/launcher.py:
   - Renders banner from DESIGN.md (exact ASCII).
   - Shows active profile, mode, subscription badge (managed only).
   - Main menu items: Remember, Recall, Search, Vault, Tokens, Agents, Settings, Quit.
   - Arrow nav via questionary.select.
   - Each choice dispatches to the corresponding typer command, re-entering typer.Cont
ext.
   - Footer: "↑↓ navigate • Enter select • q quit • ? help".

2. cli/main.py: if sys.argv is exactly ["matriosha"] and stdout is a TTY → launch
TUI; else normal typer flow.

3. Respect --json / --plain (disable TUI if set).

4. tests/test_tui_launcher.py:
   - Monkeypatch questionary to return "Quit" → launcher exits 0.
   - Non-TTY stdout → TUI not launched (regression test).

Open PR "phase6: interactive launcher".
```

Success criteria: Bare `matriosha` in a terminal shows the menu; scriptable use unaffected.

P6.2 — Branding: ASCII banner, color palette, themed components

Title: Centralize DESIGN.md visual identity into reusable rich theme + banner module.

Complexity: Simple

Input: `DESIGN.md` (full).

Output: `cli/brand/__init__.py`, `cli/brand/banner.py`, `cli/brand/theme.py`, update commands to use it.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase6/brand.

READ: DESIGN.md IN FULL (colors, typography, iconography, banner).

1. cli/brand/theme.py:
   from rich.theme import Theme
   MATRIOSHA_THEME = Theme({
       "primary": "<hex from DESIGN.md>",
       "accent":  "<hex>",
       "success": "<hex>",
       "warning": "<hex>",
       "danger":  "<hex>",
       "muted":   "dim",
       "integrity": "<hex>",
   })
   def console() -> Console: return Console(theme=MATRIOSHA_THEME)

2. cli/brand/banner.py:
   BANNER = """<exact ASCII from DESIGN.md>"""
   def print_banner(c): c.print(BANNER, style="primary")

3. Refactor all commands that print banners/panels (vault init, auth login, launcher)
to import from cli/brand.

4. Visual smoke test: tests/test_brand.py asserts banner string non-empty and theme
loadable.

Open PR "phase6: centralized brand + theme".
```

Success criteria: Consistent visual identity across every command.

P6.3 — Rich formatting layer (`cli/utils/output.py`) and `--json` / `--plain` enforcement

Title: One output helper; every command uses it; machine-readable output is schema-stable.

Complexity: Medium

Input: `DESIGN.md`, `SPECIFICATION.md` §7 (“Output format deterministic in –json”), every `cli/commands/*.py`.

Output: `cli/utils/output.py`, refactor all commands, `tests/test_output_contract.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase6/output-contract.

READ: DESIGN.md, SPECIFICATION.md §7, RULES.md.

1. cli/utils/output.py:
   class Output:
       def __init__(self, ctx: GlobalContext): ...
       def ok(self, title: str, body: dict | str = "", *, table: Table | None =
None): ...
       def warn(self, msg: str, **data): ...
       def error(self, msg: str, *, exit_code: int, **data): ...
       def json(self, payload: dict): ...   # stable json: sort_keys=True, en-
sure_ascii=False
       def plain(self, text: str): ...
   - If ctx.json_output → always emit a single JSON object per invocation; suppress ri
ch.
   - If ctx.plain → emit plain text only (no color, no tables).
   - Else → rich with theme from P6.2.

2. Refactor every command to use Output (remove direct rich.console.Console usage).

3. tests/test_output_contract.py:
   - Every major command invoked with --json produces JSON parsable by json.loads.
   - Schema snapshots (stored in tests/snapshots/*.json) for: remember, list, search,
whoami, billing status, token generate (with redacted token field).

Open PR "phase6: unified output contract".
```

Success criteria: `--json` everywhere emits valid, stable JSON; plain mode is color-free.

P6.4 — `matriosha status` and `matriosha doctor`

Title: Comprehensive diagnostics (versions, mode, config paths, connectivity, crypto self-test, vault integrity summary).

Complexity: Medium

Input: all prior phases, `SPECIFICATION.md` §3 status+doctor, `DESIGN.md`.

Output: `cli/commands/status.py`, `cli/commands/doctor.py`, `core/diagnostics.py`, `tests/test_doctor.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase6/status-doctor.

READ: SPECIFICATION.md §3 status+doctor, RULES.md §2+§3 (mode diagnostics),
DESIGN.md.

1. core/diagnostics.py collects checks (each returns CheckResult(name, status:"ok|
warn|fail", detail)):
   - Python version >= 3.11.
   - Required deps importable.
   - Config file exists + perms 0600.
   - Vault exists + unlockable (if passphrase provided via --test-passphrase or env).
   - Local vector index readable.
   - In managed mode: token valid (whoami 200), subscription active, managed endpoint
reachable (DNS + TLS).
   - Crypto self-test: encrypt/decrypt known vector.
   - Merkle self-test: known leaves → known root.
   - Time drift < 30s vs NTP (best-effort; skip on failure).

2. cli/commands/status.py: short summary table; no checks that require a passphrase.

3. cli/commands/doctor.py: runs ALL checks, with rich table of statuses + remediation
hints. Exit 0 if no fail, 10 if any fail (integrity exit). --json emits {"checks":[{na
me,status,detail,hint}]}.

4. tests/test_doctor.py:
   - All checks green on fresh install.
   - Corrupt config → doctor flags + suggests `matriosha mode set local` or similar.
   - Managed mode with no token → doctor flags auth check.

Open PR "phase6: status + doctor diagnostics".
```

Success criteria: `matriosha doctor` gives a crisp health report with actionable hints.

P6.5 — Shell completion + man page

Title: Generate bash/zsh/fish completions via `completion` command + a roff man page.

Complexity: Simple

Input: `SPECIFICATION.md` §3 completion group.

Output: `cli/commands/completion.py`, `docs/matriosha.1`, `tests/test_completion.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase6/completion-manpage.

READ: SPECIFICATION.md §3 completion.

1. cli/commands/completion.py:
   - matriosha completion <bash|zsh|fish> → prints
Typer's generated completion script (typer does this natively via Click).
   - matriosha completion install [--shell auto] → writes to appropriate location
(~/.zshrc, etc.) with a guard to avoid duplicates. Never overwrite existing unrelated
lines — append a fenced block.

2. docs/matriosha.1 (roff): synopsis, description, commands grouped per
SPECIFICATION.md §3, global flags, exit codes, files, environment, examples.

3. tests/test_completion.py: each shell variant outputs non-empty script containing co
mmand names.

Open PR "phase6: shell completion + man page".
```

Success criteria: Completion scripts install cleanly; man page renders with `man ./docs/matriosha.1`.

PHASE 7 — Testing & Documentation

P7.1 — End-to-end integration test suite

Title: Black-box CLI tests that drive the binary through full scenarios in both modes (managed mode via respx).

Complexity: Complex

Input: all prior phases.

Output: `tests/integration/` with fixtures + scenarios, update `pyproject.toml` (pytest markers `integration`), `tests/conftest.py`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase7/integration.

READ: TASKS.md T5, SPECIFICATION.md §7, RULES.md §2.

Create tests/integration/ with:
- conftest.py providing:
  * temp_home fixture (sets HOME + XDG_*_HOME to tmp_path).
  * initialized_vault fixture (runs vault init).
  * managed_profile fixture with mocked ManagedClient via respx.
  * cli_runner fixture using typer.testing.CliRunner + subprocess variant for shell co
mpletion tests.

Scenarios (each a separate test file):
- test_local_happy_path.py: init → remember 3 items → list → search → recall →
delete → verify.
- test_integrity_tamper.py: remember → corrupt payload byte on disk → vault verify --d
eep → exit 10.
- test_compress_decompress.py: full cycle.
- test_managed_sync.py: login (mocked device flow) → remember local → vault sync → se
rver has item.
- test_token_lifecycle.py: login → token generate → list → revoke → inspect.
- test_rotate_keys.py: rotate KEK then rotate data_key; verify all memories still de-
crypt.
- test_doctor_scenarios.py: green and red paths.
- test_json_contracts.py: --json output matches snapshots for core commands.
- test_mode_guards.py: managed-only commands in local mode → exit 30.

Mark with `@pytest.mark.integration`. pytest.ini_options addopts="-q -ra" and markers
= ["integration: e2e scenarios"].

Open PR "phase7: integration scenarios".
```

Success criteria: `pytest -m integration` passes with all scenarios.

P7.2 — Security audit checklist + static scans

Title: Produce `docs/SECURITY_AUDIT.md` enumerating RULES.md §2 controls + run `bandit`, `pip-audit`, `semgrep` (lightweight ruleset), and fix findings.

Complexity: Medium

Input: `RULES.md` §2 in full, all `core/` and `cli/` code.

Output: `docs/SECURITY_AUDIT.md`, CI workflow `.github/workflows/security.yml`, fixes as separate commits.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase7/security-audit.

READ: RULES.md §2 ENTIRELY.

1. Install dev tools: bandit, pip-audit, semgrep (add to [project.optional-dependen-
cies] security).
2. Run:
   - bandit -r core cli -ll
   - pip-audit
   - semgrep --config p/python --config p/owasp-top-ten core cli
3. Create docs/SECURITY_AUDIT.md:
   - For each OWASP item A01..A10 in RULES.md §2.1, map to concrete code locations + t
ests.
   - List findings from scanners with remediation notes. Fix HIGH and MEDIUM;
document accepted LOW risks.
4. Add .github/workflows/security.yml running bandit + pip-audit + semgrep on every
PR (fail on HIGH).

Open PR "phase7: security audit + CI security workflow".
```

Success criteria: Zero HIGH findings; audit doc cross-references RULES.md controls.

P7.3 — User documentation (`README.md` + `docs/USER_GUIDE.md` + `docs/COMMAND_REFERENCE.md`)

Title: Ship accurate, example-driven docs for both hardcore and vibe users.

Complexity: Medium

Input: `SPECIFICATION.md`, `DESIGN.md`, real CLI output.

Output: `README.md` (rewrite), `docs/USER_GUIDE.md`, `docs/COMMAND_REFERENCE.md`.

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase7/docs.

READ: SPECIFICATION.md, DESIGN.md, every cli/commands/*.py.

1. README.md:
   - Banner image (ASCII).
   - One-paragraph pitch (local sovereignty + managed convenience).
   - Quickstart (10 lines: install, vault init, remember, recall).
   - Two-mode explainer.
   - Security posture summary (link to SECURITY_AUDIT.md).
   - Install / dev setup.
   - License.

2. docs/USER_GUIDE.md: task-oriented walkthroughs — "Your first memory", "Syncing
across devices", "Rotating your key", "Creating an agent token", "Troubleshooting with
doctor". Each with exact copy-paste command blocks and expected output snippets.

3. docs/COMMAND_REFERENCE.md: per-command (alphabetical by group) — synopsis, flags,
examples, exit codes. Generated partly from `matriosha <group> <verb> --help`.

Tone: concise, confident, zero marketing fluff. Include a "Vibe mode" sidebar section
at the top of USER_GUIDE with the 3 commands a user really needs (init, remember,
recall).

Open PR "phase7: user documentation".
```

Success criteria: A new user can reach first-memory-remembered state from README alone.

P7.4 — Publishing preparation (PyPI + GitHub release workflow)

Title: Wheel build, version tagging, CHANGELOG, `.github/workflows/release.yml`.

Complexity: Medium

Input: `pyproject.toml`, full repo.

Output: `.github/workflows/ci.yml`, `.github/workflows/release.yml`, `CHANGELOG.md`, `.github/ISSUE_TEMPLATE/`, `LICENSE` (if missing).

Prompt:

```
Repo: /home/ubuntu/github_repos/matriosha. Branch: phase7/release.

READ: RULES.md §1 (Python-only), pyproject.toml.

1. Add LICENSE (MIT or Apache-2.0 — pick MIT unless pyproject says otherwise).

2. CHANGELOG.md following Keep-a-Changelog; seed with "2.0.0 — CLI-only rewrite" sum-
marizing Phase 1–6 deliverables.

3. .github/workflows/ci.yml:
   - On push/PR: matrix py 3.11, 3.12 on ubuntu + macos.
   - Steps: checkout, setup-python, pip install -e .[dev,security], ruff check, mypy -
-strict core cli, pytest -q, pytest -m integration.

4. .github/workflows/release.yml:
   - Trigger: tag v*.
   - Build sdist + wheel via `python -m build`.
   - Publish to PyPI via `pypa/gh-action-pypi-publish` using Trusted Publishing (OIDC
— no secrets in repo).
   - Create GitHub Release with CHANGELOG excerpt.

5. Issue templates: bug_report.md, feature_request.md, security_report.md (directs to
security@matriosha or a placeholder).

6. Add `[tool.hatch.build]` include rules to ship only core/, cli/, docs/, LICENSE, RE
ADME.md, CHANGELOG.md.

Open PR "phase7: CI + release workflow + changelog".
```

Success criteria: Tagging `v2.0.0` produces a green CI run and a PyPI-ready artifact (without actually publishing).

Appendix A — Cross-Cutting Rules (read before every session)

1. Feature branches only. Never push to `main`. Always open a PR; never merge yourself.
2. Every prompt starts with: “Read RULES.md, SPECIFICATION.md, DESIGN.md from repo root.”
3. Never invent commands outside SPECIFICATION.md §3.
4. Managed-only commands must never break local mode (RULES.md §1 + §3 isolation).
5. All secrets (passphrases, tokens, keys) must be prompted via `typer.prompt(hide_input=True)` or sourced from env/GSM — never accepted via plain CLI args in examples/docs. For managed secrets, use `core/secrets.py` with secret names (`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `STRIPE_SECRET_KEY`, etc.).
6. Exit codes are canonical (cli/utils/errors.py). Do not reuse exit 1 for anything other than fatal unknown failures.
7. `--json` output must be a single top-level JSON object, schema-stable, no trailing text.
8. Tests use pytest + typer.testing.CliRunner; no real network (respx/httpx_mock only).
9. Files containing key material: chmod 0600 on unix; on Windows document ACL reliance.
10. All user-visible strings pass through `cli/utils/output.py` so themes + –plain behave correctly.

Appendix B — Dependency Matrix Between Tasks

```
P1.1 ── P1.2 ── P1.3
                  └── P2.1 ── P2.2 ── P2.3
                                 │
                                 ├── P2.4 ── P2.5 ── P2.6 ── P2.7
                                 │                              │
                                 └── P3.1 ── P3.2 ── P3.3 ──────┤
                                                                │
                                                                ├── P4.1 ── P4.2 ──
P4.3 ── P4.4

│                               │

│                               ├── P4.5 ── P4.6

│                               │

│                               └── P5.1 ── P5.2 ── P5.3
                                                                │
                                                                └── P6.1 ── P6.2 ──
P6.3 ── P6.4 ── P6.5

               │

               └── P7.1 ── P7.2 ── P7.3 ── P7.4
```

Execute strictly in topological order. Branching sessions (e.g., P3 and P4 in parallel) is allowed if different agents work different branches, but P4.x consumes P3.1's embedder.

Appendix C — Vibe-Coder Quick Recipes

If you only want the product to work, run these in order, one prompt per session, accept the agent's PRs after review:

1. P1.1, P1.2, P1.3
2. P2.1 → P2.7 (local mode fully usable after this — you can ship open source already)
3. P3.1 → P3.3 (search + compression)
4. P6.1, P6.2, P6.3 (pretty CLI)
5. P7.1, P7.3 (tests + docs)

Then — only if monetizing — do Phase 4, 5, 6.4, 6.5, 7.2, 7.4.

— End of ATOMIC_PROMPTS.md —